# Predicting CVSS and CWE from CVE Descriptions with Fine-Tuned Small Language Models

**Tara Dixit** **Jessie Dong** **Summer Li**

Stanford University

{taradixit, jessiedong, summerli}@stanford.edu

## Abstract

Until April 2026, the National Vulnerability Database (NVD) included a Common Vulnerability Scoring System (CVSS) vector, severity tier (Low, Medium, High, or Critical) and a Common Weakness Enumeration (CWE) vulnerability type for almost every CVE. After April 2026, NIST started enriching only high priority vulnerabilities, and the overwhelming majority of new CVEs now appear without this information. We consider the task of enriching new CVEs by predicting this information: given a CVE description, predict its CVSS v3.1 base metrics, CVSS score, severity, and CWE type.

## 1 Introduction

Knowing the severity of a vulnerability and the kind of security property it violates allows one to triage the vulnerabilities to which their software is susceptible, and prioritize patches accordingly. For example, suppose one is interested in whether their software is vulnerable to buffer overflows, as in the figure on the left. The Common Weakness Enumeration (CWE) assigns each weakness a unique CWE-ID (e.g., CWE-120) and groups related CWEs hierarchically (e.g., buffer overflow weaknesses such as CWE-120 and CWE-787 are children of CWE-119), which one could leverage to answer this question. Alternatively,

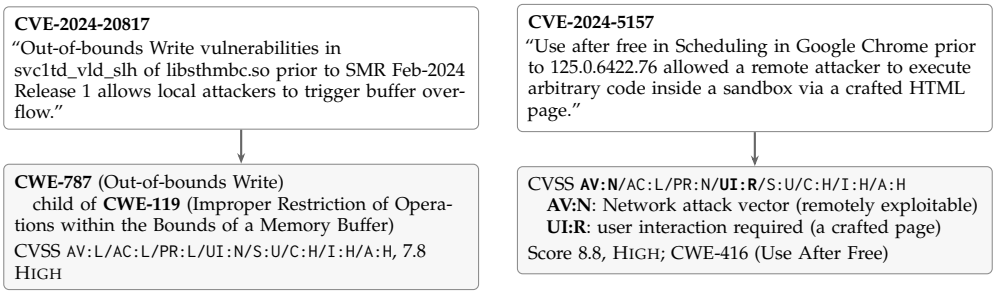



Figure 1: Two CVE descriptions from the 2024 test set and the labels NIST assigned to them. Left: an out-of-bounds write, CWE-787, which the CWE hierarchy places under the buffer-overflow class CWE-119. Right: a use-after-free in Chrome that is exploitable over the network once a user opens a crafted page.

suppose that one only wants to patch vulnerabilities that could lead to remote code execution (RCE) from untrusted websites, as in the figure on the right. The Common Vulnerability Scoring System (CVSS) provides eight components to quantify how a vulnerability is exploited and its impact on Confidentiality, Integrity, and Availability (e.g., the Network Attack Vector component in the figure, meaning remotely exploitable), and combines these components into a single base score in [0, 10] and a severity tier to ease triage.[1] These enrichments for Common Vulnerabilities and Exposures (CVE)[2] identifiers are assigned by humans at the National Vulnerability Database (NVD) maintained by the National Institute of Standards and Technology (NIST). However, due to not being able to keep up with the number of new vulnerability disclosures, a CVE backlog arose in early 2024. As such, as of 15 April 2026, NIST will enrich only CVEs present in CISA's Known Exploited Vulnerabilities (KEV) catalog or affecting US Federal government or Executive Order (EO) 14028 critical software, and mark others as "Lowest Priority – not scheduled for immediate enrichment" (National Institute of Standards and Technology, 2026). Consequently, it is important to study automated CVE enrichment.

In this paper, we pose CVE enrichment as the task of predicting the eight CVSS v3.1[3] components (FIRST, 2019) shown in Figure 1, as well as the CVSS score, severity tier, and CWE from only a text description of the vulnerability. We consider CVEs from the last three years[4] from the National Vulnerability Database, since CVEs recur, so training/test splits should account for time and train on older CVEs to avoid leakage. Specifically, within a pinned NVD release, we train on the CVEs from 2022 and 2023, and test on the 36,325 usable[5] 2024 CVEs.

**CVE enrichment.** Given only the text description of a CVE vulnerability disclosure, recover its CVSS vector and thereby its CVSS score and severity tier, as well as its CWE.

We make the following contributions:

- **Temporal benchmark for CVE enrichment.** We release a temporal benchmark for CVE enrichment free from leakage issues, based on a pinned snapshot of the NVD and validated with a reference CVSS calculator implementation. We acknowledge that our choice of ground truth, the NVD's vectors, are themselves the products of fallible human judgment: NVD analysts can and do disagree about correct vectors, reporting organizations can assign different scores to the same vulnerabilities, and NVD labels are revised after the fact. But without consensus there's no dataset, and by pinning a snapshot, at least our reported numbers won't change under us.
- **CVSS/CWE Benchmark.** We evaluate the effectiveness of (1) TF-IDF features with linear classification/regression, (2) a 3.8B-parameter Phi-3.5-mini LLM zero-shot, and (3) Phi-3.5-mini fine-tuned with LoRA at predicting (a) the eight CVSS components and deriving the score/severity from these, or (b) the CVSS score and CWE directly. We find that the fine-tuned Phi-3.5-mini achieves the best severity tier accuracy of the systems we evaluate, 69.9%,[6] and CWE accuracy of 70.9%. It matches the exact CVSS vector for 48.8% of test CVEs and outperforms the TF-IDF logistic regression baseline by +2.8 to +4.7 points across tasks ($^*$ statistically significantly w/ paired bootstrap resampling, intervals in Appendix D do not contain 0). We also find that the fine-tuned model and the TF-IDF baseline complement one another, erring on different

[1] The eight components and score determine severity: ≤3.9 LOW, ≤6.9 MEDIUM, ≤8.9 HIGH, else CRITICAL.

[2] https://www.cve.org

[3] Common Vulnerability Scoring System version 3.1.

[4] Of the NVD records that were not rejected, 94.4% of 2024 CVEs meet our inclusion criteria, against 90.7% for 2023 and 85.9% for 2022.

[5] Usable meaning containing required text description, enrichment ground truths, and having a CVSS v3.x (v3.0 or v3.1) base score.

[6] Bonhomme and Dulaunoy (2025) report over 82% severity accuracy with RoBERTa, but on a random split whose training data includes 2024 CVEs, so their numbers are not comparable to ours (Appendix C).

CVEs: at least one gets severity correct for 75.8% of test CVEs. We provide more results and discuss insights, error cases, and implications in Sections 3 and 4.

- **Previous Work.** Earlier works have used bag-of-words representations with linear models to predict CVSS components and score (Elbaz et al., 2020), BERT to predict CVSS components (Shahid and Debar, 2021), RoBERTa to predict tier (Bonhomme and Dulaunoy, 2025), and bag-of-words with random forests (Aota et al., 2020) or RoBERTa (Mosievskiy, 2026) to predict CWE. Our benchmark expands this prior work by considering a strict temporal split, more recent LLMs, and joint prediction of CVSS and CWE. We provide more related work in Section 5.
- **Reproducible Code & Artifacts.** We release our configs[7] and per-CVE predictions.[8] Our code regenerates all numbers in this work given an NVD release; we use release `v2026.10.05-000016`.[9]

## 2 Our Evaluation

**Dataset.** CVE descriptions can be very repetitive: it is common to observe several near-duplicate descriptions of the same vulnerability (e.g., found in different packages), sometimes weeks or months apart. Thus, a random split of the CVE data can produce a training set which is representative of the test data distribution. In this work, we instead take a temporal approach: train only on 2022 and 2023 CVEs and test on 2024 CVEs, to determine how well each system can generalize to future vulnerabilities. All CVE data is taken from a particular snapshot of a National Vulnerability Database (NVD) feed,[10] so as to avoid discrepancies between statistics reported in this document and results of released artifacts. We evaluate on all 36,325 2024 CVEs meeting the following inclusion criteria (see Appendix C.4 for statistics and additional information):

- CVE has a CVSS v3.x base metric vector and severity label;
- CVE has a CWE that is specific (excluding the placeholders `NVD-CWE-noinfo` and `NVD-CWE-Other`);
- CVE's NVD-provided description is in English and at least 20 characters long.

**Approach and metrics.** As discussed in the introduction, CVSS base score and severity are simple calculations (FIRST, 2019), functions of the eight different components. Thus, given a vulnerability description, we want to train systems that either (1) predict a score, or (2) predict the eight components and then calculate the score from that vector (Elbaz et al., 2020; Shahid and Debar, 2021), and analyze performance in terms of severity classification (accuracy and macro-averaged $F_1$-score), CWE classification (accuracy), and CVSS score and vector. For CVSS score, we report mean absolute error (MAE), and for CVSS vector, we report the percent of CVEs for which all eight components are correctly predicted ("Exact CVSS" in Table 1). We implemented a CVSS v3.1 calculator and verified it agrees with the CVSS base score listed by the NVD for all 86,870 CVEs in our dataset. Thus, any score errors reported are a result of mistakes in component prediction.

[7] Configs to fine-tune Phi-3.5-mini on CVSS and CWE and run all baselines on our datasets: https://github.com/jessiedong01/cvss-cwe-prediction.

[8] Predictions from all baselines for all test CVEs in JSON Lines format, where each line is a JSON object mapping field to the model's prediction.

[9] Mirror hosted by the Fraunhofer Institute for Communication, Information Processing and Ergonomics (Fraunhofer FKIE) (Fraunhofer FKIE, 2026).

[10] Release `v2026.10.05-000016` of the Fraunhofer FKIE mirror of the NVD JSON feeds (Fraunhofer FKIE, 2026).

Table 1: Results on enriching test (2024) CVEs. Systems marked ◇ predict CVSS scores by predicting vectors and computing the score using the CVSS formula. For each scoring method, bold highlights the best performing system and underline indicates second best. See Table 7 for all results.

| System | Severity acc. | Severity macro-F1 | CWE acc. | Score MAE ↓ | Exact CVSS |
|---|---|---|---|---|---|
| TF-IDF + LR | 67.1 | 47.9 | 67.2 | 0.98 | 44.1 |
| TF-IDF + CB-LR | 65.7 | 51.9 | 67.2 | 0.98 | 40.5 |
| TF-IDF + RF | 66.9 | 47.1 | 65.2 | 0.98 | 41.8 |
| Fine-tuned Phi-3.5 | **69.9** | **55.7** | **70.9** | **0.85** | **48.8** |
| TF-IDF + LR◇ | 67.1 | 51.6 | 67.2 | 0.94 | 44.1 |
| TF-IDF + CB-LR◇ | 67.1 | 53.9 | 67.2 | 0.98 | 40.5 |
| TF-IDF + RF◇ | 64.0 | 47.8 | 65.2 | 1.03 | 41.8 |
| Fine-tuned Phi-3.5◇ | **69.9** | **55.7** | **70.9** | **0.85** | **48.8** |

Table 2: Agreement between fine-tuned Phi-3.5 and TF-IDF logistic regression (formula scoring) on the 2024 test set (%). The last column is the share of CVEs on which at least one of the two is correct.

| | Both correct | Fine-tuned only | TF-IDF only | Neither | At least one |
|---|---|---|---|---|---|
| Severity | 61.3 | 8.6 | 5.9 | 24.2 | 75.8 |
| CWE | 64.4 | 6.5 | 2.9 | 26.2 | 73.8 |
| Exact CVSS vector | 39.7 | 9.1 | 4.5 | 46.7 | 53.3 |

**Systems evaluated.** For CWE classification, CVSS score, vector, and severity classification we evaluate three systems: a TF-IDF + logistic regression classifier, an instruction-tuned 3.8B-parameter LLM prompted zero-shot for classification, and that same LLM after a short (2.5 hour) LoRA fine-tuning to predict CWE and CVSS. TF-IDF-based classifiers are strong baselines on this task: Talibzade et al. (2025) find TF-IDF features perform best for CWE and severity prediction, and find them challenging to outperform on NLP tasks using vulnerability description text. Further details about data preparation and training can be found in Appendix C.

We release our code[11] as well as our system's prediction for each CVE.[12] Statistics reported here have been updated to be calculated from a particular snapshot of an NVD feed (Fraunhofer FKIE, 2026) release (`v2026.10.05-000016`); please run our provided scripts for calculation on this data release to exactly reproduce the numbers below and in Appendix D.

## 3 Findings

**Fine-tuned Phi-3.5 performs best on all tasks.** Table 1 summarizes test set performance (all 2024 CVEs in our snapshot of the dataset that have required information present: $n =$ 36,325). We find our fine-tuned Phi-3.5 model achieves the best performance, outperforming the TF-IDF + LR system by +2.8 to +4.7 points on all tasks.

**Exact CVSS.** Here, we consider the full eight-component vector, which is predicted for every test CVE (see Appendix D.4). In general, our models often predict similar outputs and severity, but Table 2 shows they are frequently in disagreement. However, for 75.8% of CVEs, at least one model is correctly predicting severity. We next examine where and how our models err in CVSS prediction.

[11] https://github.com/jessiedong01/cvss-cwe-prediction
[12] In the `results/` directory of the repository, one JSON Lines file per system.

Table 3: Recall and F1 (%) by true severity tier on the 2024 test set.

| | Critical | | High | | Medium | | Low | |
|---|---|---|---|---|---|---|---|---|
| System | Rec. | F1 | Rec. | F1 | Rec. | F1 | Rec. | F1 |
| TF-IDF + LR, direct | 50.7 | 51.4 | 62.2 | 61.1 | 77.9 | 77.1 | 1.1 | 2.2 |
| TF-IDF + LR, formula | 73.3 | 56.1 | 62.6 | 62.6 | 71.3 | 76.8 | 6.2 | 10.9 |
| TF-IDF + LR bal., formula | 57.1 | 52.8 | 61.9 | 62.2 | 75.6 | 77.2 | 19.2 | 23.2 |
| Phi-3.5 + LoRA, formula | 76.9 | 59.7 | 63.6 | 64.6 | 75.1 | 79.0 | 11.4 | 19.6 |

First, the fine-tuned model is biased towards assigning higher scores: in CVE-2024-0003, the vulnerability is rated Critical despite its CVSS vector needing higher privileges. Similarly, RoboDK in CVE-2024-0257 is scored lower by NIST at 3.3. In both examples the vulnerability descriptions are terse, and do not clearly explain the conditions which reduced their final scores. Our systems defaulted to the priors learned from similar weaknesses.

Overall, the fine-tuned model outperforms all TF-IDF systems, including ridge regression. However, there is almost no difference in performance for the fine-tuned model whether the scores are directly predicted or calculated from predicted components using the CVSS 3.1 formula. We confirm in Table 5 that the scores and severities produced by the CVSS formula from the fine-tuned model's predicted components are in fact almost always identical to the directly generated values. Thus, it is apparent from its training data, which conveniently lists the CVSS components before the score, that the fine-tuned model is already acting as though calculating scores using the formula and its generated vectors. For the logistic regression model, using formula scoring actually improves performance: score MAE decreases from 0.98 to 0.94, and macro-F1 increases from 47.9% to 51.6% (accuracy does not change at 67.1%). This is partly due to the increased Critical recall (50.7% $\rightarrow$ 73.3%) as seen in Table 3, resulting from predicting Critical for 7,755 CVEs in comparison to 4,800 in the NVD. As such, Critical F1 increases from 51.4% to 56.1%, along with a substantial improvement in Low F1 (2.2% $\rightarrow$ 10.9%). This comes at the cost of reduced Critical precision (52.1% $\rightarrow$ 45.4%). On the other hand, random forest's comparatively worse per component performance negatively influences its formula predicted scores (66.9% $\rightarrow$ 64.0% in accuracy and 0.98 $\rightarrow$ 1.03 MAE).

The Low severity tier is the most difficult for all systems, likely because it represents less than 2% of our training data. As shown in Figure 2, most other errors involve predicting adjacent severity levels. It is encouraging that using class-balanced training weights greatly improves Low tier recall to 19.2% in the TF-IDF model. The fine-tuned model performed reasonably well in this under-represented tier (recall of 11.4%) even without data rebalancing.

Figure 3 shows per-CVSS-component accuracy on the 2024 test data, excluding the base score and severity which are dependent on the other components. All models perform poorly on privileges required, with TF-IDF and the fine-tuned model at 76.5% and 78.8% respectively. Attack complexity, scope, attack vector, and user interaction are likely easier to classify because they are almost always explicitly specified in the vulnerability description, and indeed, all systems perform well with above 90% accuracy. The next most difficult components after privileges required are the confidentiality, integrity, and availability impact scores, where Marchiori et al. (2025) found LLMs underperform embedding classifiers. By contrast, we find that our fine-tuned model sees its largest improvements over TF-IDF on the less keyword dependent privileges required and CIA components, for instance, accuracy increases on the confidentiality and integrity components are 82.9% $\rightarrow$ 84.8% and 82.9% $\rightarrow$ 84.5% respectively.

Figure 4 shows CWE accuracy on test set CVEs against how often each NVD-assigned CWE weakness appears in training. Our TF-IDF model was limited to predicting on the

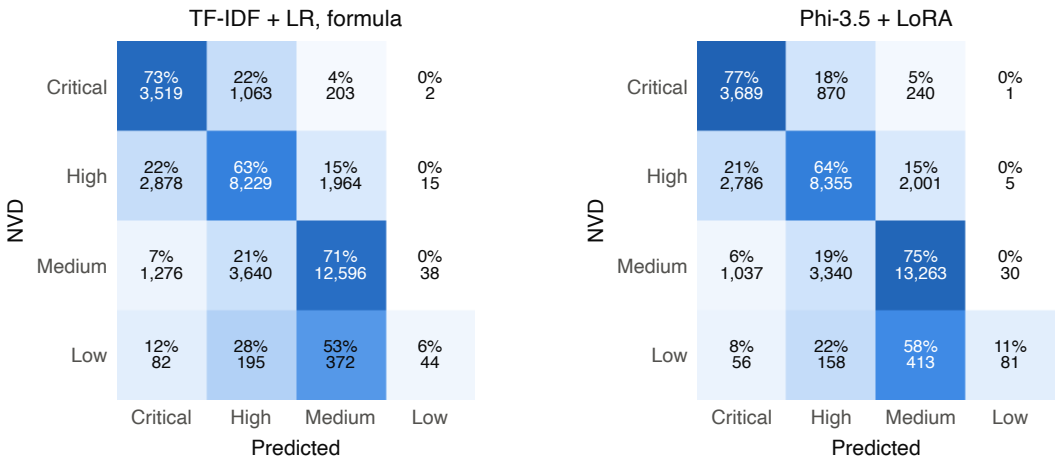



Figure 2: Severity confusion matrices on the 2024 test set, normalized by true tier, for TF-IDF logistic regression with formula scoring (left) and fine-tuned Phi-3.5 (right).

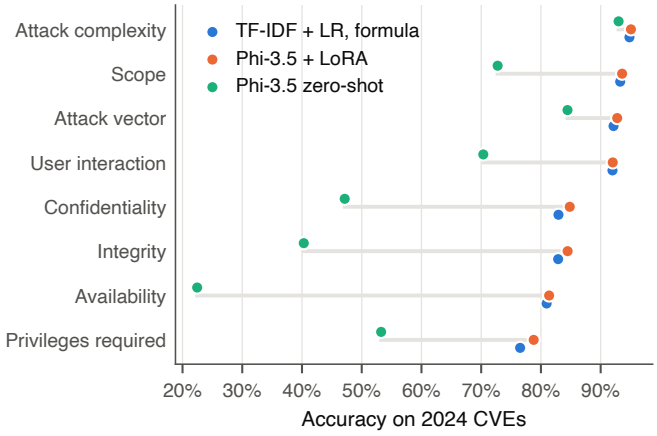



Figure 3: Accuracy per CVSS component on 2024 CVEs. TF-IDF and the fine-tuned model are scored on the full test set, the zero-shot model on its 2,000-CVE sample.

100 most common CWEs present in the training set, whereas the language model may predict an arbitrary textual CWE. Accordingly, our TF-IDF model scores 0% accuracy on the 3,023 CVEs with less commonly represented CWEs, while the fine-tuned model predicts 7.0% correctly. As expected, weaknesses that are common, and likely named explicitly in the vulnerability description, like cross-site scripting or SQL injection vulnerabilities, are near perfectly predicted by both of our trained models. However, for less common weaknesses, or when the description is more general in nature, it appears our models learned associations between certain weaknesses and the severity metrics.

In Figure 5, both TF-IDF and fine-tuned models over-predict the most common critical score more often than it is actually seen, likely also due to learned associations between weaknesses and common CVSS scores assigned to them. By contrast, ridge regression predicts many scores unseen in the NVD data and does not predict the most critical or benign CVSS scores as frequently as they actually occur. On the 2024 test set, logistic regression with formula scoring predicts a score of 9.8 (the most common critical score) on 17.7% of CVEs, in comparison to 10.5% in the NVD. This bias results in a mean signed error of +0.33, compared to only +0.08 for the ridge regression model which produces a more continuous score distribution.

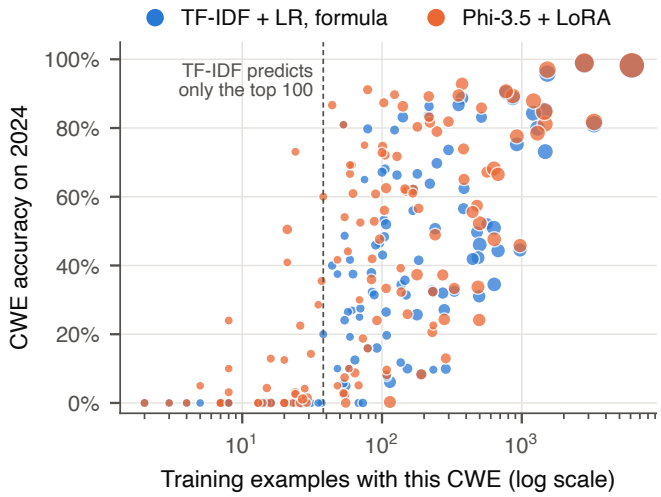



Figure 4: CWE accuracy on the 2024 test set against the number of training examples with each CWE, for every CWE with at least 20 test CVEs. Point size grows with test count.

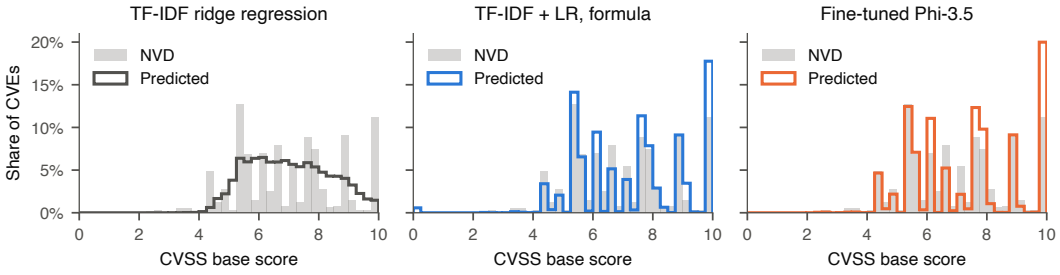



Figure 5: Distribution of predicted and NVD base scores on the 2024 test set in bins of 0.25, for ridge regression (left), TF-IDF logistic regression with formula scoring (middle), and the fine-tuned model (right).

To understand more deeply where and why models disagree with NVD-assigned vectors, we qualitatively analyzed errors. Results are shown in Tables 18 through 21: for each CVSS component, we chose four example CVEs from our 2024 test data where all (Tables 18 and 19) or at least one (Tables 20 and 21) of our systems erred. We bolded portions of the NVD description which may have served as clues for the true value. In many cases only a single component differs between our predictions and the true value, and therefore exact match accuracy under-reports relative understanding of CVSS vectors.

We find the same general tendencies with the fine-tuned model as Marchiori et al. (2025) did with larger LLMs, namely, that some CVSS v3.1 components can be learned better than others. Table 9 compares the frequency of the possible values per CVSS v3.1 component that NIST assigned with those predicted by our model. For example, the model overpredicts CVSS scores of 9.8 (20.0% instead of 10.5%). And correspondingly, it labels 7,568 of the CVEs from the test set as Critical, in comparison to 4,800 CVEs labeled Critical in the NVD. Thus, 48.7% of those CVEs the model considers Critical are indeed Critical according to the NVD. Overall, the model behaves similarly to the formula-scored logistic regression, meaning it is positively biased (mean signed error of +0.33).

When analyzing the predicted vectors, we find that in 48.8% of the test CVEs, all vector components are predicted correctly (TF-IDF: 44.1%). In 25.5% only one component is incorrect (TF-IDF: 28.1%), in 12.7% two (TF-IDF: 14.1%) and in 13.0% at least three (TF-IDF: 13.6%). Thus, in many cases where our method misses, only one single detail is incorrectly

Table 4: Comparison on the same random sample of 2,000 test CVEs, on which every system is scored.

| | Severity | | CWE | Score | Vector | |
|---|---|---|---|---|---|---|
| System | Acc. | Macro-F1 | Acc. | MAE ↓ | Partial | Exact |
| TF-IDF + LR, formula | 65.3 ±2.1 | 47.6 ±1.8 | 66.2 ±2.2 | 0.99 ±0.06 | 86.5 ±0.7 | 42.2 ±2.2 |
| Phi-3.5 zero-shot, direct | 20.8 ±1.7 | 15.4 ±2.4 | 25.7 ±2.0 | 1.58 ±0.05 | 60.5 ±0.7 | 2.4 ±0.6 |
| Phi-3.5 zero-shot, formula | 43.5 ±2.3 | 26.8 ±1.7 | 25.7 ±2.0 | 2.08 ±0.08 | 60.5 ±0.7 | 2.4 ±0.6 |
| Phi-3.5 + LoRA, formula | **69.6** ±2.0 | **52.8** ±3.3 | **68.8** ±2.1 | **0.87** ±0.05 | **87.5** ±0.7 | **48.0** ±2.1 |

predicted. Often times, that detail may be hard to guess as it may be left out in the CVE description, like how exactly a DoS manifests itself (Conf. high or just avail.). Also, the conditions for privilege requirements may be subtle to understand. Overall, we can conclude that even the small fine-tuned language model significantly outperforms the classical baselines at learning how NIST enriches CVE descriptions with CVSS vectors, although the performance increase is not as big as the models sizes suggest it to be.

Table 4 shows the predictions made by our zero-shot setup. First of all, in 99.7% of the cases the answer could be parsed as JSON object (from a random sample of 2,000 CVEs, 11 contained a valid JSON format but one of the CVSS components was invalid, e.g., a missing Scope value or Availability=CRITICAL). We refer to Table 15 for more details. The table compares the distribution of CVSS severity written by the LLM to those that NIST derived for the same CVEs. Clearly, the distribution is skewed towards Critical: 62.4% of the CVEs are predicted Critical (NVD: 13.3%) and accuracy for severity prediction is only 20.8%. Medium is underpredicted (9 CVEs, or 0.4%, instead of 47.5%).

Furthermore, we find that the written CVSS score is the result of the written CVSS vector in only 3.4% of the CVEs (see Table 5). This may be due to the difficulty of performing arithmetic in context. In fact, even for humans, CVSS calculation is considered difficult and tools exist to perform the computation. We find similar problems when looking at the severity column of Table 5: here, we compare the severity label written by the LLM to the severity implied by the written score. Only for 37.0% they match. Using the severity implied by the written vector instead increases the zero-shot severity accuracy on a random sample of 2,000 test CVEs from 20.8% (using the written severity label) to 43.5%. For comparison, on the same CVEs, fine-tuning achieves 69.6% and the TF-IDF method 65.3% severity accuracy. The MAE of the scores worsens for zero-shot from 1.58 (using the written score) to 2.08 when instead calculating the score from the predicted CVSS vector. Thus, the written scores are on average closer to the ground truth than those implied by the predicted CVSS vectors.

Table 5: Self-consistency of language-model outputs (%), where *Score = formula* is the share of outputs whose written score equals the CVSS v3.1 score of the written vector and *Severity = tier of score* is the share whose written severity matches the tier of the written score.

| Model | Valid JSON | Score = formula | Severity = tier of score |
|---|---|---|---|
| Phi-3.5 zero-shot | 99.7 | 3.4 | 37.0 |
| Phi-3.5 + LoRA | 100.0 | 99.8 | 100.0 |

## 4 Discussion

In the previous sections, we have already observed a couple of strengths and weaknesses in the predictions our methods make. We further inspect common mistakes and compare

our results.

**Common Mistakes in CVSS Prediction.** As can be seen in Figure 3 and Tables 20 and 21, certain CVSS vector components are much easier to predict than others, no matter the method used for prediction. Attack complexity and attack vector can be predicted with an accuracy of 84% to 93% zero-shot, whereas impact to availability, integrity and confidentiality are much more difficult to predict correctly (22% to 47% zero-shot).

Furthermore, it becomes apparent from the tables that models tend to predict a CVSS vector for the weakness type mentioned in the CVE description if little further information is available or if the limiting factors are not mentioned or difficult to extract.

**Training and Run Times.** We find that severity classification accuracy (Table 16) increases per checkpoint. We validated on 1,000 CVEs from the validation split for each checkpoint (every 1,000 steps, up to 5,000 training steps). While accuracy increases up to step 5,000, Macro-$F_1$ scores peak earlier (optimum at 4,000 steps with a score of 62.1%). Depending on whether mispredictions of the rarer classes Critical and Low are considered more important to reduce than increasing overall accuracy, the model at step 4,000 might be preferred for deployment.

Fine-tuning takes 2.5 h, scoring the test set (one year's worth of CVEs) another 4.9 h on our local machine (an Apple M5 Pro with 64 GB memory). Batches contained at most 32 CVE descriptions. For comparison, all TF-IDF experiments, including tuning, took only 9 minutes on an Apple M5 Pro.

Next, we investigate which CVSS vector components models struggle to predict correctly. Table 10 presents the distribution of CVEs in which a given CVSS vector component is predicted incorrectly, for CVEs with one incorrect vector component. We present this both for our fine-tuned model and for TF-IDF as a representative classical method. Privileges Required makes up 38% of such examples for the fine-tuned model and TF-IDF respectively, with Availability Impact making up 17% and 16% respectively.

Finally, we investigate what causes the difference between CVEs with correct severity versus exact CVSS score match. To our surprise, most of the gap can be attributed to CVEs with only one incorrect component (see Table 11). Thus, often a single ambiguous word can determine whether a given vulnerability is Critical or High, for example. Two examples are presented in Table 12. In the case of CVE-2024-0003, it appears NIST inferred from the phrasing *administrative service* that this is not easily exploitable by outsiders and gave it a score of 7.2 requiring high privileges. However, the phrasing *malicious user* led the model to predict an attacker does not need privileges, resulting in a Critical score of 9.8. All other vector components were predicted correctly. In the case of CVE-2024-0008, NVD gave a High score of 8.8 as User Interaction was deemed required to exploit the vulnerability. However, no mention is made of any user in the text, leading the model to make an incorrect prediction and give a score of 9.8. Here again, all other components were predicted correctly. In both cases, one could argue that the model's interpretation is reasonable, given the missing details.

Next we examine MAE of the predicted score by true severity (Table 13). On the 2024 test set, our fine-tuned model's score MAE is 0.53 for Critical CVEs, 0.76 on High, 0.92 on Medium, and 2.90 on Low CVEs. Low severity CVEs are rare, making accurate prediction challenging for all methods. Interestingly, Ridge without vector commitment achieves a score MAE of 2.91 on Low CVEs. When our model errs on a Low CVE it is usually by more than a severity tier. Two examples are given in Table 12. In both cases, the descriptions contain words associated with particularly insidious weakness types. The first CVE describes a heap overflow, a weakness type our model has likely seen associated with high severity during training, and makes a prediction accordingly. However, the last sentence mitigates the severity, and NVD scored this accordingly. The second CVE also vaguely alludes to a

common weakness our model likely associates with high severity, where session-handling code can easily be construed as an authentication bypass vulnerability. More information would be needed to clarify that this is likely less serious. Thus, when the description alludes to a weakness that our model associates with serious vulnerabilities, but otherwise offers no relevant detail, the model appears to predict according to the weakness, rather than the sentence. Low severity performs better for TF-IDF with class-balanced weights. The analogue in a language model would be oversampling Low during training, which we did not explore.

We next analyze model performance as a function of the length of the CVE Description, split into quartiles 1…4 from shortest to longest (with bounds 194, 285, and 444 on the 2024 test set). As shown in Table 14, model performance is lower when there is less text to work with (severity accuracy on the shortest CVEs being 61.7% and 59.7% for our fine-tuned model and TF-IDF respectively), with improvement and leveling off as description length increases (74.4% on the third quartile of lengths and 73.9% on the fourth for our model, with a similar trend for exact vector). Exact vector match results show the same trends as severity. Many of the shortest descriptions are simply of the form PRODUCTNAME + weakness type, e.g. CVE-2024-0057 is titled *.NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability*, and the predicted score would be entirely dependent on the learned associations of Microsoft and SFBO vulnerabilities. While the extra capacity of the larger fine-tuned model may be helpful overall, it has a similar advantage over TF-IDF in all quartiles, and thus does not appear to overcome poor performance on very short CVE descriptions.

## 5 Related Work

**Automatic vulnerability triage.** Elbaz et al. (2020) considered the problem of triaging new vulnerabilities as they are disclosed, and asked which triage decisions an analyst could automate using only the information available at that time. For predicting the eight base CVSS metrics from vulnerability descriptions represented as bags of words, they found a linear classifier to perform best. They report both directly predicting CVSS scores, and predicting CVSS vectors from which to compute CVSS scores. CVSS-BERT (Shahid and Debar, 2021) takes a similar approach, training a BERT classifier for each metric, and use the specification's formula to compute CVSS scores from predicted vectors. Marchiori et al. (2025) find that a classifier over strong embeddings outperforms general-purpose LLMs, at least on the three impact-related CVSS metrics. VLAI (Bonhomme and Dulaunoy, 2025) fine-tunes RoBERTa on more than 600,000 vulnerabilities to predict the severity tier. VLAI was trained on more than 600,000 vulnerabilities including CVEs from 2024 and later years, meaning we cannot fairly compare to this system on our test set.

For CWE prediction, Aota et al. (2020) predict using random forest classification over bag-of-words features. More recently, Mosievskiy (2026) show that a small (125M parameter) RoBERTa language model fine-tuned on CVE-CWE pairs performs on par with larger LLMs for this task. Talibzade et al. (2025) find that for both CWE and severity prediction, TF-IDF features outperform larger LLMs prompted generically about the task, which they hypothesize is due to the highly schematic nature of CVE descriptions. We find similar results on zero-shot severity prediction in Section 3.

CVE-LLM (Ghosh et al., 2025) fine-tunes on vulnerabilities in medical devices, using a domain ontology for supervision, and Torkamani et al. (2025) use LLMs to predict CWE and severity as part of a vulnerability triage tool.

**LLMs for vulnerability-related tasks.** Recent work has considered LLMs on a range of vulnerability-related tasks. VulnWatch (Databricks, 2025) uses LLMs to prioritize vulnerabilities. Roytman (2025) finds that, for predicting likely exploitation (EPSS scores), even the

best performing frontier LLMs lag behind state of the art statistical models, while costing orders of magnitude more to query at scale.

Unlike VulnWatch (Databricks, 2025) and much previous work, we run our models on local hardware and evaluate on vulnerabilities introduced strictly after those in our training data.

## 6 Limitations

Our CWE evaluation using descriptions alone is likely to underestimate achievable performance, since CWE prediction is much easier when, unlike us but like NVD analysts, you can draw on context outside the description (e.g., by reading the original advisory). This point bears out in the CWEs predicted incorrectly according to the NVD in Table 12, where the CVSS vectors are nonetheless correct. (More CWE disagreements between the model and the NVD appear in Section 4.) For CVE-2024-0204 (an authentication bypass in GoAnywhere MFT), the NVD chooses CWE-425 (direct request, also called forced browsing), while our model chooses CWE-287 (improper authentication), arguably a more intuitive CWE. For CVE-2024-0149 (a vulnerability in an NVIDIA driver allowing "unauthorized access to files" and a "limited information disclosure"), the NVD chooses CWE-125 (out-of-bounds read), while our model chooses CWE-200 (exposure of sensitive information); it's true that the information is sensitive, but the description makes no mention of an out-of-bounds read. Some fraction of the CWE errors reported in Table 7 stem from such disagreements between the text and the NVD's CWE labels, and several of the other errors in Section 4 are alternative interpretations a human might endorse.

Our benchmark evaluates each model on a single (held out) year's worth of test CVEs. Moreover, each model is trained just once. Accordingly, the error bars reported in Appendix D do not reflect variability from training, but only from randomly sampling test-year CVEs.

For space reasons, we only report the performance of our zero-shot LLM on a random sample of 2,000 test CVEs, not on the entire year as for other models. As such, the confidence intervals reported in Table 4 do not reflect the size of our full test set, but rather the 2,000-CVE subset.

## 7 Conclusion

Our results suggest that CVE enrichment is tractable with modest computational resources, using only CVE description text. Our TF-IDF logistic regression model trains in just a few minutes, requires no GPU, and trails the fine-tuned LLM by just a few points in each CVSS component and CWE. Moreover, our simple formula scoring, which costs nothing, rescues the TF-IDF model's rare tiers; we therefore recommend deploying formula scoring on top of TF-IDF classification in all but applications that prize precision over recall on Critical vulnerabilities. The 3.8B fine-tuned model trains in just 2.5 hours, and achieves better results across the board: more consistent severity scores and CVSS vectors, and CWEs outside the frequent CWEs. At inference time, it takes on the order of a half second to predict each CVE. If either throughput or accuracy were insufficient, we might train for more than one epoch (validation accuracy was still climbing; see Table 16), oversample or weight the loss of Low predictions (class-balanced weights did help our TF-IDF model), or try larger open models amenable to fine-tuning. We leave these directions to future work.

We did not evaluate CVSS v4.0 vectors, which are starting to appear for new CVEs. We expect formula scoring to work with these vectors too, though we have not tested it. Our fine-tuned LLM could perhaps be retrained on the new format.

Our zero-shot LLM performs poorly by comparison. We do not recommend using it for CVE enrichment.

Practitioners might wonder how best to use these models. While we did not train or evaluate an ensemble, both our trained models make errors on mostly different CVEs; in principle, then, they might achieve up to 75.8% accuracy on tiers if ensembled somehow (e.g., by prediction agreement or model confidence). An ensemble along these lines might or might not come close to this upper bound; we leave such investigations to future work. Until then, we would be loath to deploy either trained model without bearing in mind that their Low tiers are unreliable, as is the privileges required component of CVSS vectors. A CVE should be checked before a predicted 9.8 score moves it to the front of the triage queue.

## References


Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024.

Masaki Aota, Hideaki Kanehara, Masaki Kubo, Noboru Murata, Bo Sun, and Takeshi Takahashi. Automation of vulnerability classification from its description using machine learning. In *IEEE Symposium on Computers and Communications (ISCC)*, 2020.

Cédric Bonhomme and Alexandre Dulaunoy. VLAI: A RoBERTa-based model for automated vulnerability severity classification, 2025.

Databricks. VulnWatch: AI-enhanced prioritization of vulnerabilities. https://www.databricks.com/blog/vulnwatch-ai-enhanced-prioritization-vulnerabilities, 2025.

Clément Elbaz, Louis Rilling, and Christine Morin. Fighting n-day vulnerabilities with automated CVSS vector prediction at disclosure. In *Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES)*, 2020. doi: 10.1145/3407023.3407038.

FIRST. Common vulnerability scoring system v3.1: Specification document. https://www.first.org/cvss/v3.1/specification-document, 2019.

Fraunhofer FKIE. NVD JSON data feeds, release v2026.10.05-000016. https://github.com/fkie-cad/nvd-json-data-feeds, 2026.

Rikhiya Ghosh, Hans-Martin von Stockhausen, Martin Schmitt, George Marica Vasile, Sanjeev Kumar Karn, and Oladimeji Farri. CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025. doi: 10.1609/aaai.v39i28.35139.

Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. https://github.com/ml-explore/mlx, 2023.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Francesco Marchiori, Denis Donadel, and Mauro Conti. Can LLMs classify CVEs? investigating LLMs capabilities in computing CVSS vectors. In *TrustAICyberSec Workshop*, 2025.

Nikita Mosievskiy. Fine-tuning RoBERTa for CVE-to-CWE classification: A 125M parameter model competitive with LLMs, 2026.

National Institute of Standards and Technology. NIST updates NVD operations to address record CVE growth. https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth, April 2026. Accessed 2026-10-05.

Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Michael Roytman. Benchmarking LLMs on the vulnerability prioritization task. Empirical Security. https://research.empiricalsecurity.com/research/benchmarking-llms, 2025.

Mustafizur R. Shahid and Hervé Debar. CVSS-BERT: Explainable natural language processing to determine the severity of a computer security vulnerability from its description. In *20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2021.

Rustam Talibzade, Idilio Drago, and Francesco Bergadano. On using LLMs for vulnerability classification. In *Proceedings of the 2025 Workshop on Large AI Systems and Models with Privacy and Security Analysis (LAMPS)*, 2025. doi: 10.1145/3733800.3763267.

Mohammad Jalili Torkamani, Joey NG, Nikita Mehrotra, Mahinthan Chandramohan, Padmanabhan Krishnan, and Rahul Purandare. Streamlining security vulnerability triage with large language models, 2025.

## A Prompts

Both prompts are sent as a single user turn through the Phi-3.5 chat template, and descriptions longer than 2,000 characters (1.4% of test CVEs) are cut at that length.

**Fine-tuned model (user turn).**

```
You are a cybersecurity expert. Given the CVE description below, predict the
NVD enrichment fields.
Return ONLY a JSON object with no extra text.

CVE Description:
<CVE description>
```

**Zero-shot model (user turn).**

```
You are a cybersecurity expert specializing in vulnerability analysis. Given a
CVE description, predict the NVD CVSS v3.1 enrichment fields.

CVE Description:
<CVE description>

Respond ONLY with one JSON object with exactly these keys and allowed values:
{"attack_vector": "NETWORK"|"ADJACENT_NETWORK"|"LOCAL"|"PHYSICAL",
"attack_complexity": "LOW"|"HIGH", "privileges_required": "NONE"|"LOW"|"HIGH",
"user_interaction": "NONE"|"REQUIRED", "scope": "UNCHANGED"|"CHANGED",
"confidentiality": "NONE"|"LOW"|"HIGH", "integrity": "NONE"|"LOW"|"HIGH",
"availability": "NONE"|"LOW"|"HIGH", "cwe": "CWE-<number>", "cvss_score":
<number 0.0-10.0>, "severity": "CRITICAL"|"HIGH"|"MEDIUM"|"LOW"}
```

## B Reproducibility

All code, configuration files, and per-CVE predictions are in the project repository, the NVD snapshot is pinned to FKIE release `v2026.10.05-000016`, and each step below is a single command that regenerates its part of the paper.

- Data preparation with `download_data.py` and `prepare_splits.py`.
- TF-IDF baselines with `baselines_classical.py`.
- LoRA training with `mlx_lm.lora` and `scripts/lora_phi35.yaml`.
- Checkpoint selection with `select_checkpoint.py`.
- Evaluation with `evaluate_lora.py`.
- Tables, figures, and numbers in the text with `make_tables.py` and `plot_results.py`.

# C Data and Training Details

## C.1 Data

We use the publicly available National Vulnerability Database (NVD)[13] for vulnerability data. Up until recently the NVD enriched nearly all CVEs with CWEs, CVSS base scores, severity labels and more, but since April 2026 (National Institute of Standards and Technology, 2026) it only enriches vulnerabilities it considers high-priority.

We include all CVEs with a CWE, primary[14] base CVSS v3 score and severity, published from 2022 to 2024 (both included), for a total of 86,870. Of these, 776 have CVSS v3.0 scores, and the rest v3.1. The formula for computing the base score from the eight components gives the same score in both versions for every one of these CVEs.

For training/validation/test we use a temporal split, to prevent data leakage and better reflect real use: We train and validate on older CVEs, and test on newer ones. The 50,545 CVEs with identifiers from 2022 and 2023 are split into training and validation data randomly with an 80/20 ratio, stratified by severity, resulting in 40,436 training and 10,109 validation CVEs, Table 6. The test data are the 36,325 remaining (i.e., usable) CVEs, all with identifiers from 2024. Note that Low severity is less than 2% of all CVEs, in all splits. The CWEs exhibit a long tail, Table 6. There are 464 different CWEs in the training set and 512 in the test set, but the top 100 CWEs in the training set occur in 91.7% of the test CVEs. We do not use the recent VLAI data and model of Bonhomme and Dulaunoy (2025) because the random train/validation/test split of the underlying dataset, `CIRCL/vulnerability-scores`, includes CVEs from 2024 (also present in our test data), 2025 and 2026, so VLAI's performance could in part reflect memorisation rather than generalisation ability. A temporal split is necessary to fairly compare all systems' abilities to generalise, particularly since products mentioned in descriptions repeat often.

## C.2 TF-IDF Baselines

As baselines we implement TF-IDF classification/regression methods similar to previous work (Elbaz et al., 2020). We create TF-IDF representations with unigrams and bigrams, using sublinear tf and min document frequency 2, including stop words since words such as "not" or "without" can affect vulnerability meaning. We limit the number of features to 50,000. We train a different model for each output, Table 1, using scikit-learn (Pedregosa et al., 2011):

**LogReg** We use logistic regression, and select the regularisation strength, $C \in \{0.3, 1, 3, 10\}$, based on validation set accuracy, separately for severity and CWE classification, and for base component classification when predicting all eight components at once.

[13] https://nvd.nist.gov

[14] When a CVE has several CVSS entries, we use the primary score assigned by NIST.

**CB-LogReg** Class-balanced logistic regression: Identical setup, except we weight each example inversely proportional to its true class frequency.

**Rand. Forest** Untuned random forest classifier, with 300 trees for severity and CWE, or 100 for each base component.

**Ridge** Untuned ridge regression to predict scores, except that we choose the $\alpha \in \{0.3, 1, 3, 10\}$ with lowest MAE in validation.

For CWE we only train the models on the most frequent 100 CWEs in the training data, plus an OTHER class for the remaining CWEs. A prediction of OTHER is always considered an error since we compare with the CVEs' true CWEs.

### C.3 Language Model

We use the 3.8B-parameter phi-3.5-mini-instruct (Phi) causal LM (decoder-only LLM) (Abdin et al., 2024). Unlike VulnWatch (Databricks, 2025), a system that utilises hosted (commercial) LLMs, Phi is available for download, and specialised enough, to be practically usable on local hardware. Phi's training data is reported to end in October 2023,[15] i.e., before any CVEs in the test set.

We fine-tune with LoRA (Hu et al., 2022) using MLX (Hannun et al., 2023), on a single Apple M5 Pro chip with 64 GB unified memory. The LoRAs (rank 16, scale 20, dropout 0.05) are applied to all linear projections in the final 16 transformer layers, totalling 12.6M parameters, only 0.329% of Phi's parameters. We train for 5,000 steps (one epoch) with batch size 8. The learning rate is $10^{-4}$ for the first 100 (warm-up) steps, then decays cosinally to $10^{-5}$. We exclude 561 examples (1.4%) with more than 768 tokens to limit memory use. In contrast we include all test set CVEs, irrespective of length.

The fine-tuning target is a single JSON object: First the eight base score components, then CWE, score, and severity. Placing score after components ensures that Phi's output is internally consistent, since it has already (essentially)[16] committed to a vector by the time it outputs the score, similar to our formula scoring method (Section 2). We evaluate a saved model every 1,000 training steps on a sample of 1,000 validation CVEs, scoring predicted scores via formula, and keep the one with best severity accuracy using greedy decoding, Table 16.

For zero-shot Phi we use a prompt with a brief task description, all output field names and the legal values for components and severity (Appendix A). Zero-shot Phi is very verbose and slow, so to limit run time we only include it on a random sample of 2,000 CVEs from the test set. Where relevant, we evaluate other systems on the same sample.

### C.4 Dataset Statistics

Table 6: Data splits, with severity columns giving the share of CVEs (%) in each tier.

| Split | CVEs | Critical | High | Medium | Low | CWEs |
|---|---|---|---|---|---|---|
| Train (2022–23) | 40,436 | 15.4 | 39.3 | 43.7 | 1.7 | 464 |
| Validation (2022–23) | 10,109 | 15.4 | 39.3 | 43.7 | 1.7 | 293 |
| Test (2024) | 36,325 | 13.2 | 36.2 | 48.6 | 1.9 | 512 |

[15] https://huggingface.co/microsoft/Phi-3.5-mini-instruct

[16] Generation is not constrained, so the model can still write a score that disagrees with its vector. It does so on less than 1% of test CVEs (Table 5).

# D Full Results

## D.1 All Systems

Table 7: All systems on the full 2024 test set (36,325 CVEs). Values are percentages except score MAE. ± gives the half-width of the 95% bootstrap interval. Bold marks the best value in each column. The TF-IDF direct systems share one ridge regression, so their score MAE is identical.

| | Severity | | CWE | Score | Vector | |
|---|---|---|---|---|---|---|
| System | Acc. | Macro-F1 | Acc. | MAE ↓ | Partial | Exact |
| TF-IDF + LR, direct | 67.1 ±0.5 | 47.9 ±0.6 | 67.2 ±0.5 | 0.98 ±0.01 | 86.9 ±0.2 | 44.1 ±0.5 |
| TF-IDF + LR, formula | 67.1 ±0.5 | 51.6 ±0.8 | 67.2 ±0.5 | 0.94 ±0.01 | 86.9 ±0.2 | 44.1 ±0.5 |
| TF-IDF + LR balanced, direct | 65.7 ±0.5 | 51.9 ±0.8 | 67.2 ±0.5 | 0.98 ±0.01 | 86.0 ±0.2 | 40.5 ±0.5 |
| TF-IDF + LR balanced, formula | 67.1 ±0.5 | 53.9 ±0.9 | 67.2 ±0.5 | 0.98 ±0.01 | 86.0 ±0.2 | 40.5 ±0.5 |
| TF-IDF + RF, direct | 66.9 ±0.5 | 47.1 ±0.6 | 65.2 ±0.5 | 0.98 ±0.01 | 86.1 ±0.2 | 41.8 ±0.5 |
| TF-IDF + RF, formula | 64.0 ±0.5 | 47.8 ±0.6 | 65.2 ±0.5 | 1.03 ±0.01 | 86.1 ±0.2 | 41.8 ±0.5 |
| Phi-3.5 + LoRA, direct | **69.9** ±0.5 | **55.7** ±0.9 | **70.9** ±0.5 | **0.85** ±0.01 | **87.9** ±0.2 | **48.8** ±0.5 |
| Phi-3.5 + LoRA, formula | **69.9** ±0.5 | **55.7** ±0.9 | **70.9** ±0.5 | **0.85** ±0.01 | **87.9** ±0.2 | **48.8** ±0.5 |

## D.2 Paired Differences

Table 8: Fine-tuned Phi-3.5 minus TF-IDF logistic regression with formula scoring, on the same 36,325 test CVEs, with 95% paired bootstrap intervals.

| Metric | Difference | 95% interval |
|---|---|---|
| Severity accuracy (points) | +2.8 | [+2.4, +3.1] |
| Severity macro-F1 (points) | +4.1 | [+3.0, +5.2] |
| CWE accuracy (points) | +3.7 | [+3.4, +4.0] |
| Score MAE | -0.09 | [-0.10, -0.08] |
| Exact CVSS vector (points) | +4.7 | [+4.3, +5.0] |

### D.3 Component Values

Table 9: Share of 2024 test CVEs (%) with each value of each CVSS v3.1 component, as assigned by NIST and as predicted by the fine-tuned model.

| Component | Value | NVD | Fine-tuned |
|---|---|---|---|
| Attack vector | Network | 76.2 | 79.2 |
| | Adjacent | 2.9 | 0.9 |
| | Local | 20.1 | 19.5 |
| | Physical | 0.7 | 0.4 |
| Attack complexity | Low | 94.3 | 98.2 |
| | High | 5.7 | 1.8 |
| Privileges required | Low | 40.3 | 35.2 |
| | High | 9.4 | 4.7 |
| | None | 50.4 | 60.1 |
| User interaction | None | 66.1 | 64.1 |
| | Required | 33.9 | 35.9 |
| Scope | Unchanged | 73.5 | 76.6 |
| | Changed | 26.5 | 23.4 |
| Confidentiality | Low | 30.6 | 25.2 |
| | High | 48.8 | 56.0 |
| | None | 20.6 | 18.8 |
| Integrity | Low | 32.1 | 27.8 |
| | High | 41.2 | 45.9 |
| | None | 26.7 | 26.3 |
| Availability | Low | 9.0 | 0.2 |
| | High | 48.8 | 55.3 |
| | None | 42.2 | 44.5 |

### D.4 Exact CVSS Vectors

Table 10: Which component is wrong when exactly one of the eight is wrong (% of such CVEs), for the fine-tuned model and TF-IDF logistic regression on the 2024 test set.

| Component | Fine-tuned | TF-IDF |
|---|---|---|
| Attack vector | 7 | 7 |
| Attack complexity | 6 | 4 |
| Privileges required | 38 | 38 |
| User interaction | 12 | 10 |
| Scope | 5 | 5 |
| Confidentiality | 10 | 12 |
| Integrity | 6 | 8 |
| Availability | 17 | 16 |

Table 11: CVEs whose severity is predicted correctly, split by how many of the eight vector components are wrong (% of all 2024 test CVEs). Severity can be right while the vector is not, and most such CVEs have exactly one wrong component.

| | Severity correct | 0 wrong | 1 wrong | 2 wrong | 3+ wrong |
|---|---|---|---|---|---|
| Fine-tuned Phi-3.5 | 69.9 | 48.8 | 13.1 | 4.9 | 3.2 |
| TF-IDF + LR, formula | 67.1 | 44.1 | 14.6 | 5.3 | 3.1 |

## E Additional Analysis

Table 12: Test CVEs discussed in Section 4, with the NVD labels and the output of the fine-tuned model. The last column lists the vector components the model got wrong.

| CVE | Description | NVD | Predicted | Wrong |
|---|---|---|---|---|
| CVE-2024-0003 | A condition exists in FlashArray Purity whereby a malicious user could use a remote administrative service to create an account on the array allowing privileged access. | 7.2 High, CWE-269 | 9.8 Critical, CWE-287 | PR |
| CVE-2024-0008 | Web sessions in the management interface in Palo Alto Networks PAN-OS software do not expire in certain situations, making it susceptible to unauthorized access. | 8.8 High, CWE-613 | 9.8 Critical, CWE-613 | UI |
| CVE-2024-0257 | RoboDK v5.5.4 is vulnerable to heap-based buffer overflow while processing a specific project file. The resulting memory corruption may crash the application. | 3.3 Low, CWE-122 | 7.8 High, CWE-787 | C, I, A |
| CVE-2024-10214 | Mattermost versions 9.11.X <= 9.11.1, 9.5.x <= 9.5.9 icorrectly issues two sessions when using desktop SSO - one in the browser and one in desktop with incorrect settings. | 3.5 Low, CWE-303 | 9.8 Critical, CWE-287 | PR, UI, C, I, A |
| CVE-2024-0204 | Authentication bypass in Fortra's GoAnywhere MFT prior to 7.4.1 allows an unauthorized user to create an admin user via the administration portal. | 9.8 Critical, CWE-425 | 9.8 Critical, CWE-287 | none |
| CVE-2024-0149 | NVIDIA GPU Display Driver for Linux contains a vulnerability which could allow an attacker unauthorized access to files. A successful exploit of this vulnerability might lead to limited information disclosure. | 3.3 Low, CWE-125 | 3.3 Low, CWE-200 | none |

Table 13: Mean absolute score error by true severity tier on the 2024 test set.

| System | Critical | High | Medium | Low |
|---|---|---|---|---|
| TF-IDF ridge regression (direct) | 1.23 | 0.80 | 0.96 | 2.91 |
| TF-IDF + LR, formula | 0.59 | 0.81 | 1.04 | 3.26 |
| Phi-3.5 + LoRA, formula | 0.53 | 0.76 | 0.92 | 2.90 |

Table 14: Severity accuracy and exact-vector rate (%) by quartile of description length on the 2024 test set. Quartile boundaries are 194, 285, and 444 characters.

| | Shortest quartile | | Second | | Third | | Longest quartile | |
|---|---|---|---|---|---|---|---|---|
| System | Sev. | Vector | Sev. | Vector | Sev. | Vector | Sev. | Vector |
| TF-IDF + LR, formula | 59.7 | 38.8 | 67.2 | 38.4 | 71.0 | 50.7 | 70.6 | 48.7 |
| Phi-3.5 + LoRA, formula | 61.7 | 42.0 | 69.7 | 42.0 | 74.4 | 55.7 | 73.9 | 55.5 |

Table 15: Severity tiers on the 2,000-CVE zero-shot sample: as assigned by NIST, as written by zero-shot Phi-3.5, and as computed from the vector it wrote. Counts with shares (%) in parentheses.

| Tier | NVD | Zero-shot, written | Zero-shot, from vector |
|---|---|---|---|
| Critical | 266 (13.3) | 1,247 (62.4) | 68 (3.4) |
| High | 742 (37.1) | 697 (34.8) | 558 (27.9) |
| Medium | 951 (47.5) | 9 (0.4) | 1,146 (57.3) |
| Low | 41 (2.1) | 2 (0.1) | 15 (0.8) |
| No valid tier | – | 45 (2.2) | – |

Table 16: Validation metrics of the fine-tuned model under formula scoring at each saved checkpoint, on the same 1,000 validation CVEs.

| Step | Severity acc. | Macro-F1 | CWE acc. | Score MAE | Exact vector |
|---|---|---|---|---|---|
| 1,000 | 68.2 | 50.8 | 65.4 | 0.83 | 52.9 |
| 2,000 | 72.6 | 56.9 | 67.2 | 0.73 | 55.4 |
| 3,000 | 74.4 | 58.5 | 71.1 | 0.66 | 62.2 |
| 4,000 | 76.0 | 62.1 | 73.4 | 0.65 | 63.0 |
| 5,000 | 76.3 | 58.6 | 74.2 | 0.63 | 63.9 |

Table 17: Wall-clock time on a single Apple M5 Pro (64 GB), with inference run in small batches of at most 32 CVEs and without optimization for speed.

| Step | Time |
|---|---|
| All TF-IDF baselines, tuning and test (CPU) | 9 min |
| LoRA fine-tuning, 5,000 steps | 2.5 h |
| LoRA inference, 36,325 test CVEs | 4.9 h |

Table 18: Test CVEs on which all four trained systems predicted the wrong value for the listed component (attack vector, attack complexity, privileges required, and user interaction). Bold marks words in the NVD description that may signal the true value. FT: fine-tuned Phi-3.5, LR: logistic regression, CB: class-balanced logistic regression, RF: random forest, all with formula scoring. Values: N network or none, A adjacent, L local or low, P physical, H high, R required, U unchanged, C changed.

| Component | CVE | Description | True | Predicted |
|---|---|---|---|---|
| Attack vector | CVE-2024-0001 | A condition exists in FlashArray Purity whereby a local account intended for initial array configuration remains active potentially allowing a malicious actor to gain elevated privileges. | N | FT:L LR:L CB:L RF:L |
| | CVE-2024-0311 | A malicious insider can bypass the existing policy of Skyhigh Client Proxy without a valid release code. | L | FT:N LR:N CB:N RF:N |
| | CVE-2024-0315 | Remote file inclusion vulnerability in FireEye Central Management affecting version 9.1.1.956704. This vulnerability allows an attacker to upload a malicious PDF file to the system during the report creation process. | L | FT:N LR:N CB:N RF:N |
| | CVE-2024-0321 | Stack-based Buffer Overflow in GitHub repository gpac/gpac prior to 2.3-DEV. | N | FT:L LR:L CB:L RF:L |
| Attack complexity | CVE-2024-0056 | Microsoft.Data.SqlClient and System.Data.SqlClient SQL Data Provider Security Feature Bypass Vulnerability | H | FT:L LR:L CB:L RF:L |
| | CVE-2024-0144 | NVIDIA nvJPEG2000 library contains a vulnerability where an attacker can cause a buffer overflow issue by means of a specially crafted JPEG2000 file. A successful exploit of this vulnerability might lead to data tampering. | H | FT:L LR:L CB:L RF:L |
| | CVE-2024-0410 | An authorization bypass vulnerability was discovered in GitLab affecting versions 15.1 prior to 16.7.6, 16.8 prior to 16.8.3, and 16.9 prior to 16.9.1. A developer could bypass CODEOWNERS approvals by creating a merge conflict. | H | FT:L LR:L CB:L RF:L |
| | CVE-2024-0980 | The Auto-update service for Okta Verify for Windows is vulnerable to two flaws which in combination could be used to execute arbitrary code. | H | FT:L LR:L CB:L RF:L |
| Privileges required | CVE-2024-0179 | SMM Callout vulnerability within the AmdCpmDisplayFeatureSMM driver could allow locally authenticated attackers to overwrite SMRAM, potentially resulting in arbitrary code execution. | H | FT:L LR:L CB:L RF:L |
| | CVE-2024-0231 | A resource misdirection vulnerability in GitLab CE/EE versions 12.0 prior to 17.0.5, 17.1 prior to 17.1.3, and 17.2 prior to 17.2.1 allows an attacker to craft a repository import in such a way as to misdirect commits. | H | FT:L LR:N CB:N RF:N |
| | CVE-2024-0311 | A malicious insider can bypass the existing policy of Skyhigh Client Proxy without a valid release code. | L | FT:N LR:N CB:N RF:N |
| | CVE-2024-0315 | Remote file inclusion vulnerability in FireEye Central Management affecting version 9.1.1.956704. This vulnerability allows an attacker to upload a malicious PDF file to the system during the report creation process. | L | FT:N LR:N CB:N RF:N |
| User interaction | CVE-2024-0008 | Web sessions in the management interface in Palo Alto Networks PAN-OS software do not expire in certain situations, making it susceptible to unauthorized access. | R | FT:N LR:N CB:N RF:N |
| | CVE-2024-0321 | Stack-based Buffer Overflow in GitHub repository gpac/gpac prior to 2.3-DEV. | N | FT:R LR:R CB:R RF:R |
| | CVE-2024-0322 | Out-of-bounds Read in GitHub repository gpac/gpac prior to 2.3-DEV. | N | FT:R LR:R CB:R RF:R |
| | CVE-2024-0410 | An authorization bypass vulnerability was discovered in GitLab affecting versions 15.1 prior to 16.7.6, 16.8 prior to 16.8.3, and 16.9 prior to 16.9.1. A developer could bypass CODEOWNERS approvals by creating a merge conflict. | R | FT:N LR:N CB:N RF:N |

Table 19: Test CVEs on which all four trained systems predicted the wrong value for the listed component (scope and the three impacts). Bold marks words in the NVD description that may signal the true value. FT: fine-tuned Phi-3.5, LR: logistic regression, CB: class-balanced logistic regression, RF: random forest, all with formula scoring. Values: N network or none, A adjacent, L local or low, P physical, H high, R required, U unchanged, C changed.

| Component | CVE | Description | True | Predicted |
|---|---|---|---|---|
| Scope | CVE-2024-0056 | Microsoft.Data.SqlClient and System.Data.SqlClient SQL Data Provider Security Feature Bypass Vulnerability | C | FT:U LR:U CB:U RF:U |
| | CVE-2024-0171 | Dell PowerEdge Server BIOS contains an TOCTOU race condition vulnerability. A local low privileged attacker could potentially exploit this vulnerability to gain access to otherwise unauthorized resources. | C | FT:U LR:U CB:U RF:U |
| | CVE-2024-0179 | SMM Callout vulnerability within the AmdCpmDisplayFeatureSMM driver could allow locally authenticated attackers to overwrite SMRAM, potentially resulting in arbitrary code execution. | C | FT:U LR:U CB:U RF:U |
| | CVE-2024-0410 | An authorization bypass vulnerability was discovered in GitLab affecting versions 15.1 prior to 16.7.6, 16.8 prior to 16.8.3, and 16.9 prior to 16.9.1. A developer could bypass CODEOWNERS approvals by creating a merge conflict. | C | FT:U LR:U CB:U RF:U |
| Confidentiality | CVE-2024-0171 | Dell PowerEdge Server BIOS contains an TOCTOU race condition vulnerability. A local low privileged attacker could potentially exploit this vulnerability to gain access to otherwise unauthorized resources. | L | FT:H LR:H CB:H RF:H |
| | CVE-2024-0257 | RoboDK v5.5.4 is vulnerable to heap-based buffer overflow while processing a specific project file. The resulting memory corruption may **crash** the application. | N | FT:H LR:H CB:H RF:H |
| | CVE-2024-0403 | Recipes version 1.5.10 allows arbitrary HTTP requests to be made through the server. This is possible because the application is vulnerable to SSRF. | L | FT:H LR:H CB:N RF:H |
| | CVE-2024-0677 | The Pz-LinkCard WordPress plugin through 2.5.1 does not prevent users from pinging arbitrary hosts via some of its shortcodes, which could allow high privilege users such as contributors to perform SSRF attacks. | L | FT:H LR:H CB:H RF:H |
| Integrity | CVE-2024-0171 | Dell PowerEdge Server BIOS contains an TOCTOU race condition vulnerability. A local low privileged attacker could potentially exploit this vulnerability to gain access to otherwise unauthorized resources. | L | FT:H LR:H CB:H RF:H |
| | CVE-2024-0257 | RoboDK v5.5.4 is vulnerable to heap-based buffer overflow while processing a specific project file. The resulting memory corruption may **crash** the application. | N | FT:H LR:H CB:H RF:H |
| | CVE-2024-0741 | An out of bounds write in ANGLE could have allowed an attacker to corrupt memory leading to a potentially exploitable **crash**. This vulnerability affects Firefox < 122, Firefox ESR < 115.7, and Thunderbird < 115.7. | N | FT:H LR:H CB:H RF:H |
| | CVE-2024-0749 | A phishing site could have repurposed an 'about:' dialog to show phishing content with an incorrect origin in the address bar. This vulnerability affects Firefox < 122 and Thunderbird < 115.7. | L | FT:H LR:H CB:H RF:H |
| Availability | CVE-2024-0009 | An improper verification vulnerability in the GlobalProtect gateway feature of Palo Alto Networks PAN-OS software enables a malicious user with stolen credentials to establish a VPN connection from an unauthorized IP address. | L | FT:H LR:N CB:N RF:N |
| | CVE-2024-0068 | Improper Link Resolution Before File Access ('Link Following') vulnerability in HYPR Workforce Access on MacOS allows File Manipulation.This issue affects Workforce Access: before 8.7.1. | N | FT:H LR:H CB:H RF:H |
| | CVE-2024-0160 | Dell Client Platform contains an incorrect authorization vulnerability. An attacker with physical access to the system could potentially exploit this vulnerability by bypassing BIOS authorization to modify settings in the BIOS. | H | FT:N LR:N CB:N RF:N |
| | CVE-2024-0171 | Dell PowerEdge Server BIOS contains an TOCTOU race condition vulnerability. A local low privileged attacker could potentially exploit this vulnerability to gain access to otherwise unauthorized resources. | L | FT:H LR:H CB:H RF:H |

Table 20: Test CVEs on which at least one, but not all, of the four trained systems predicted the wrong value for the listed component (attack vector, attack complexity, privileges required, and user interaction). Bold marks words in the NVD description that may signal the true value. FT: fine-tuned Phi-3.5, LR: logistic regression, CB: class-balanced logistic regression, RF: random forest, all with formula scoring. Values: N network or none, A adjacent, L local or low, P physical, H high, R required, U unchanged, C changed.

| Component | CVE | Description | True | Predicted |
|---|---|---|---|---|
| Attack vector | CVE-2024-0092 | NVIDIA GPU Driver for Windows and Linux contains a vulnerability where an improper check or improper handling of exception conditions might lead to denial of service. | L | FT:L LR:L CB:P RF:L |
| | CVE-2024-0100 | NVIDIA Triton Inference Server for Linux contains a vulnerability in the tracing API, where a user can corrupt system files. A successful exploit of this vulnerability might lead to denial of service and data tampering. | N | FT:N LR:N CB:L RF:N |
| | CVE-2024-0144 | NVIDIA nvJPEG2000 library contains a vulnerability where an attacker can cause a buffer overflow issue by means of a specially crafted JPEG2000 file. A successful exploit of this vulnerability might lead to data tampering. | N | FT:L LR:L CB:L RF:N |
| | CVE-2024-0147 | NVIDIA GPU display driver for Windows and Linux contains a vulnerability where referencing memory after it has been freed can lead to denial of service or data tampering. | L | FT:L LR:L CB:P RF:L |
| Attack complexity | CVE-2024-0057 | NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability | L | FT:L LR:L CB:H RF:L |
| | CVE-2024-0179 | SMM Callout vulnerability within the AmdCpmDisplayFeatureSMM driver could allow locally authenticated attackers to overwrite SMRAM, potentially resulting in arbitrary code execution. | L | FT:L LR:L CB:H RF:L |
| | CVE-2024-10253 | A potential TOCTOU vulnerability was reported in PC Manager, Lenovo Browser, and Lenovo App Store that could allow a local attacker to cause a system crash. | H | FT:H LR:L CB:L RF:H |
| | CVE-2024-10397 | A malicious server can crash the OpenAFS cache manager and other client utilities, and possibly execute arbitrary code. | L | FT:L LR:L CB:H RF:L |
| Privileges required | CVE-2024-0001 | A condition exists in FlashArray Purity whereby a local account intended for initial array configuration remains active potentially allowing a malicious actor to gain elevated privileges. | N | FT:L LR:N CB:N RF:L |
| | CVE-2024-0002 | A condition exists in FlashArray Purity whereby an attacker can employ a privileged account allowing remote access to the array. | N | FT:H LR:N CB:N RF:N |
| | CVE-2024-0003 | A condition exists in FlashArray Purity whereby a malicious user could use a remote **administrative** service to create an account on the array allowing privileged access. | H | FT:N LR:L CB:H RF:L |
| | CVE-2024-0004 | A condition exists in FlashArray Purity whereby an user with array **admin** role can execute arbitrary commands remotely to escalate privilege on the array. | H | FT:H LR:H CB:H RF:L |
| User interaction | CVE-2024-0057 | NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability | N | FT:N LR:R CB:R RF:N |
| | CVE-2024-0144 | NVIDIA nvJPEG2000 library contains a vulnerability where an attacker can cause a buffer overflow issue by means of a specially crafted JPEG2000 file. A successful exploit of this vulnerability might lead to data tampering. | R | FT:R LR:N CB:N RF:N |
| | CVE-2024-0207 | HTTP3 dissector crash in Wireshark 4.2.0 allows denial of service via packet injection or crafted capture file | N | FT:R LR:R CB:R RF:N |
| | CVE-2024-0208 | GVCP dissector crash in Wireshark 4.2.0, 4.0.0 to 4.0.11, and 3.6.0 to 3.6.19 allows denial of service via packet injection or crafted capture file | N | FT:R LR:R CB:R RF:N |

Table 21: Test CVEs on which at least one, but not all, of the four trained systems predicted the wrong value for the listed component (scope and the three impacts). Bold marks words in the NVD description that may signal the true value. FT: fine-tuned Phi-3.5, LR: logistic regression, CB: class-balanced logistic regression, RF: random forest, all with formula scoring. Values: N network or none, A adjacent, L local or low, P physical, H high, R required, U unchanged, C changed.

| Component | CVE | Description | True | Predicted |
|---|---|---|---|---|
| Scope | CVE-2024-0078 | NVIDIA GPU Display Driver for Windows and Linux contains a vulnerability in the kernel mode layer, where a user in a **guest** can cause a NULL-pointer dereference in the host, which may lead to denial of service. | C | FT:U LR:U CB:C RF:U |
| | CVE-2024-0315 | Remote file inclusion vulnerability in FireEye Central Management affecting version 9.1.1.956704. This vulnerability allows an attacker to upload a malicious PDF file to the system during the report creation process. | U | FT:U LR:U CB:C RF:U |
| | CVE-2024-0427 | The ARForms - Premium WordPress Form Builder Plugin WordPress plugin before 6.4.1 does not properly escape user-controlled input when it is reflected in some of its AJAX actions. | U | FT:C LR:U CB:U RF:U |
| | CVE-2024-0440 | Attacker, with permission to submit a link or submits a link via POST to be collected that is using the file:// protocol can then introspect host files and other relatively stored files. | U | FT:U LR:U CB:C RF:U |
| Confidentiality | CVE-2024-0008 | Web sessions in the management interface in Palo Alto Networks PAN-OS software do not expire in certain situations, making it susceptible to unauthorized access. | H | FT:H LR:N CB:N RF:H |
| | CVE-2024-0009 | An improper verification vulnerability in the GlobalProtect gateway feature of Palo Alto Networks PAN-OS software enables a malicious user with stolen credentials to establish a VPN connection from an unauthorized IP address. | L | FT:H LR:N CB:L RF:H |
| | CVE-2024-0057 | NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability | H | FT:H LR:H CB:L RF:H |
| | CVE-2024-0074 | NVIDIA GPU Display Driver for Linux contains a vulnerability where an attacker may access a memory location after the end of the buffer. A successful exploit of this vulnerability may lead to **denial of service** and data tampering. | N | FT:H LR:N CB:N RF:N |
| Integrity | CVE-2024-0008 | Web sessions in the management interface in Palo Alto Networks PAN-OS software do not expire in certain situations, making it susceptible to unauthorized access. | H | FT:H LR:N CB:N RF:H |
| | CVE-2024-0009 | An improper verification vulnerability in the GlobalProtect gateway feature of Palo Alto Networks PAN-OS software enables a malicious user with stolen credentials to establish a VPN connection from an unauthorized IP address. | L | FT:H LR:L CB:L RF:H |
| | CVE-2024-0056 | Microsoft.Data.SqlClient and System.Data.SqlClient SQL Data Provider Security Feature Bypass Vulnerability | H | FT:H LR:N CB:N RF:H |
| | CVE-2024-0057 | NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability | H | FT:H LR:L CB:L RF:H |
| Availability | CVE-2024-0008 | Web sessions in the management interface in Palo Alto Networks PAN-OS software do not expire in certain situations, making it susceptible to unauthorized access. | H | FT:H LR:N CB:N RF:N |
| | CVE-2024-0056 | Microsoft.Data.SqlClient and System.Data.SqlClient SQL Data Provider Security Feature Bypass Vulnerability | N | FT:H LR:N CB:N RF:H |
| | CVE-2024-0057 | NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability | H | FT:H LR:N CB:L RF:H |
| | CVE-2024-0144 | NVIDIA nvJPEG2000 library contains a vulnerability where an attacker can cause a buffer overflow issue by means of a specially crafted JPEG2000 file. A successful exploit of this vulnerability might lead to data tampering. | H | FT:N LR:H CB:H RF:H |